CM3: Les fonctions

TI102 - Algorithmique

Rado Rakotonarivo

rado.rakotonarivo@efrei.fr

Définition

Contexte

- Souvent un algorithme traitant un problème complexe est découpé en plusieurs parties.

Définition

Contexte

- Souvent un algorithme traitant un problème complexe est découpé en plusieurs parties.
  - Afin organiser les étapes de résolution

Définition

Contexte

- Souvent un algorithme traitant un problème complexe est découpé en plusieurs parties.
  - Afin organiser les étapes de résolution
  - Éventuellement pour travailler à plusieurs sur un même problème

Définition

Contexte

- Souvent un algorithme traitant un problème complexe est découpé en plusieurs parties.
  - Afin organiser les étapes de résolution
  - Éventuellement pour travailler à plusieurs sur un même problème
  - Pour faciliter la lecture et la compréhension de la solution

Définition

Contexte

- Souvent un algorithme traitant un problème complexe est découpé en plusieurs parties.
  - Afin organiser les étapes de résolution
  - Éventuellement pour travailler à plusieurs sur un même problème
  - Pour faciliter la lecture et la compréhension de la solution
- Ces parties sont souvent appelées **fonctions**.

Définition

Contexte

- Souvent un algorithme traitant un problème complexe est découpé en plusieurs parties.
  - Afin organiser les étapes de résolution
  - Éventuellement pour travailler à plusieurs sur un même problème
  - Pour faciliter la lecture et la compréhension de la solution
- Ces parties sont souvent appelées **fonctions**.

Définition

Contexte

- Souvent un algorithme traitant un problème complexe est découpé en plusieurs parties.
  - Afin organiser les étapes de résolution
  - Éventuellement pour travailler à plusieurs sur un même problème
  - Pour faciliter la lecture et la compréhension de la solution
- Ces parties sont souvent appelées **fonctions**.

Définition:

Une fonction est:

- Une suite d'instructions qui permet de réaliser une tâche spécifique.

Définition

Contexte

- Souvent un algorithme traitant un problème complexe est découpé en plusieurs parties.
  - Afin organiser les étapes de résolution
  - Éventuellement pour travailler à plusieurs sur un même problème
  - Pour faciliter la lecture et la compréhension de la solution
- Ces parties sont souvent appelées **fonctions**.

Définition:

Une fonction est:

- Une suite d'instructions qui permet de réaliser une tâche spécifique.
- Un nom qu'on donne à un calcul ou une suite d'instructions.

Définition

Contexte

- Souvent un algorithme traitant un problème complexe est découpé en plusieurs parties.
  - Afin organiser les étapes de résolution
  - Éventuellement pour travailler à plusieurs sur un même problème
  - Pour faciliter la lecture et la compréhension de la solution
- Ces parties sont souvent appelées **fonctions**.

Définition:

Une fonction est:

- Une suite d'instructions qui permet de réaliser une tâche spécifique.
- Un nom qu'on donne à un calcul ou une suite d'instructions.
- Un bloc d'instructions qui peut être *appelé* par un autre bloc d'instructions.

Définition

Caractéristiques d'une fonction

Il existe 3 notions importantes concernant les fonctions:

- La **déclaration** ou le **prototype** de la fonction permet de faire connaître l'existence de la fonction et de renseigner les informations la concernant:

Définition

Caractéristiques d'une fonction

Il existe 3 notions importantes concernant les fonctions:

- La **déclaration** ou le **prototype** de la fonction permet de faire connaître l'existence de la fonction et de renseigner les informations la concernant:
  - son *nom*,

Définition

Caractéristiques d'une fonction

Il existe 3 notions importantes concernant les fonctions:

- La **déclaration** ou le **prototype** de la fonction permet de faire connaître l'existence de la fonction et de renseigner les informations la concernant:
  - son *nom*,
  - ses *paramètres d'entrées*,

Définition

Caractéristiques d'une fonction

Il existe 3 notions importantes concernant les fonctions:

- La **déclaration** ou le **prototype** de la fonction permet de faire connaître l'existence de la fonction et de renseigner les informations la concernant:
  - son *nom*,
  - ses *paramètres d'entrées*,
  - ses *paramètres de sorties*

Définition

Caractéristiques d'une fonction

Il existe 3 notions importantes concernant les fonctions:

- La **déclaration** ou le **prototype** de la fonction permet de faire connaître l'existence de la fonction et de renseigner les informations la concernant:
  - son *nom*,
  - ses *paramètres d'entrées*,
  - ses *paramètres de sorties*
- La **définition** de la fonction, c'est la description de l'ensemble des instructions que la fonction aura a exécuter lors de son appel

Définition

Caractéristiques d'une fonction

Il existe 3 notions importantes concernant les fonctions:

- La **déclaration** ou le **prototype** de la fonction permet de faire connaître l'existence de la fonction et de renseigner les informations la concernant:
  - son *nom*,
  - ses *paramètres d'entrées*,
  - ses *paramètres de sorties*
- La **définition** de la fonction, c'est la description de l'ensemble des instructions que la fonction aura a exécuter lors de son appel
- L'**appel** de la fonction, c'est l'utilisation proprement dite de la fonction

Pourquoi les fonctions ?

Problème 1

On appelle amplitude d'une série L de k valeurs la différence entre la plus grande valeur et la plus petite valeur de cette série. Pour une série de valeurs entières donnée, calculer son amplitude.

Pourquoi les fonctions ?

Problème 1

On appelle amplitude d'une série L de k valeurs la différence entre la plus grande valeur et la plus petite valeur de cette série. Pour une série de valeurs entières donnée, calculer son amplitude.

Solution

Afin de résoudre ce problème, on peut détailler le raisonnement comme suit:

1. Trouver le maximum de la série L

Pourquoi les fonctions ?

Problème 1

On appelle amplitude d'une série L de k valeurs la différence entre la plus grande valeur et la plus petite valeur de cette série. Pour une série de valeurs entières donnée, calculer son amplitude.

Solution

Afin de résoudre ce problème, on peut détailler le raisonnement comme suit:

1. Trouver le maximum de la série L
2. Trouver le minimum de la série L

Pourquoi les fonctions ?

Problème 1

On appelle amplitude d'une série L de k valeurs la différence entre la plus grande valeur et la plus petite valeur de cette série. Pour une série de valeurs entières donnée, calculer son amplitude.

Solution

Afin de résoudre ce problème, on peut détailler le raisonnement comme suit:

1. Trouver le maximum de la série L
2. Trouver le minimum de la série L
3. Calculer la différence entre les deux

Pourquoi les fonctions ?

Problème 1

On appelle amplitude d'une série L de k valeurs la différence entre la plus grande valeur et la plus petite valeur de cette série. Pour une série de valeurs entières donnée, calculer son amplitude.

Solution

Afin de résoudre ce problème, on peut détailler le raisonnement comme suit:

1. Trouver le maximum de la série L au moyen d'une fonction $\max(L, k)$

Pourquoi les fonctions ?

Problème 1

On appelle amplitude d'une série L de k valeurs la différence entre la plus grande valeur et la plus petite valeur de cette série. Pour une série de valeurs entières donnée, calculer son amplitude.

Solution

Afin de résoudre ce problème, on peut détailler le raisonnement comme suit:

1. Trouver le maximum de la série L au moyen d'une fonction $\max(L, k)$
2. Trouver le minimum de la série L au moyen d'une fonction $\min(L, k)$

Pourquoi les fonctions ?

Problème 1

On appelle amplitude d'une série L de k valeurs la différence entre la plus grande valeur et la plus petite valeur de cette série. Pour une série de valeurs entières donnée, calculer son amplitude.

Solution

Afin de résoudre ce problème, on peut détailler le raisonnement comme suit:

1. Trouver le maximum de la série L au moyen d'une fonction $\max(L, k)$
2. Trouver le minimum de la série L au moyen d'une fonction $\min(L, k)$
3. Calculer la différence entre les deux en calculant $\max(L, k) - \min(L, k)$

Pourquoi les fonctions ?

Problème 1

On appelle amplitude d'une série L de k valeurs la différence entre la plus grande valeur et la plus petite valeur de cette série. Pour une série de valeurs entières donnée, calculer son amplitude.

Solution

Afin de résoudre ce problème, on peut détailler le raisonnement comme suit:

1. Trouver le maximum de la série L au moyen d'une fonction $\max(L, k)$
2. Trouver le minimum de la série L au moyen d'une fonction $\min(L, k)$
3. Calculer la différence entre les deux en calculant $\max(L, k) - \min(L, k)$

Pourquoi les fonctions ?

Problème 1

On appelle amplitude d'une série L de k valeurs la différence entre la plus grande valeur et la plus petite valeur de cette série. Pour une série de valeurs entières donnée, calculer son amplitude.

Solution

Afin de résoudre ce problème, on peut détailler le raisonnement comme suit:

1. Trouver le maximum de la série L au moyen d'une fonction $\max(L, k)$
2. Trouver le minimum de la série L au moyen d'une fonction $\min(L, k)$
3. Calculer la différence entre les deux en calculant $\max(L, k) - \min(L, k)$

Les fonctions servent à découper un problème

Les fonctions permettent de découper un problème complexe en sous-problèmes plus simples

Pourquoi les fonctions ?

Problème 2

On considère à présent n séries $L_1, L_2, \ldots, L_n$ de valeurs entières ayant respectivement $k_1, k_2, \ldots, k_n$ valeurs. On souhaite calculer l'amplitude maximale parmi ces n séries.

Pourquoi les fonctions ?

Problème 2

On considère à présent n séries $L_1, L_2, \ldots, L_n$ de valeurs entières ayant respectivement $k_1, k_2, \ldots, k_n$ valeurs. On souhaite calculer l'amplitude maximale parmi ces n séries.

Solution

Afin de résoudre ce problème, on peut détailler le raisonnement comme suit:

1. Calculer l'amplitude de chaque série L_i avec $1 \leq i \leq n$ calculer $\max(L_i, k_i) - \min(L_i, k_i)$

Pourquoi les fonctions ?

Problème 2

On considère à présent n séries $L_1, L_2, \ldots, L_n$ de valeurs entières ayant respectivement $k_1, k_2, \ldots, k_n$ valeurs. On souhaite calculer l'amplitude maximale parmi ces n séries.

Solution

Afin de résoudre ce problème, on peut détailler le raisonnement comme suit:

1. Calculer l'amplitude de chaque série L_i avec $1 \leq i \leq n$ calculer $\max(L_i, k_i) - \min(L_i, k_i)$
2. Stocker chaque amplitude dans une nouvelle série R de n valeurs

Pourquoi les fonctions ?

Problème 2

On considère à présent n séries $L_1, L_2, \ldots, L_n$ de valeurs entières ayant respectivement $k_1, k_2, \ldots, k_n$ valeurs. On souhaite calculer l'amplitude maximale parmi ces n séries.

Solution

Afin de résoudre ce problème, on peut détailler le raisonnement comme suit:

1. Calculer l'amplitude de chaque série L_i avec $1 \leq i \leq n$ calculer $\max(L_i, k_i) - \min(L_i, k_i)$
2. Stocker chaque amplitude dans une nouvelle série R de n valeurs
3. Calculer l'amplitude maximale de la série R au moyen de la fonction $\max(R, n)$

Pourquoi les fonctions ?

Problème 2

On considère à présent n séries $L_1, L_2, \ldots, L_n$ de valeurs entières ayant respectivement $k_1, k_2, \ldots, k_n$ valeurs. On souhaite calculer l'amplitude maximale parmi ces n séries.

Solution

Afin de résoudre ce problème, on peut détailler le raisonnement comme suit:

1. Calculer l'amplitude de chaque série L_i avec $1 \leq i \leq n$ calculer $\max(L_i, k_i) - \min(L_i, k_i)$
2. Stocker chaque amplitude dans une nouvelle série R de n valeurs
3. Calculer l'amplitude maximale de la série R au moyen de la fonction $\max(R, n)$

Pourquoi les fonctions ?

Problème 2

On considère à présent n séries $L_1, L_2, \ldots, L_n$ de valeurs entières ayant respectivement $k_1, k_2, \ldots, k_n$ valeurs. On souhaite calculer l'amplitude maximale parmi ces n séries.

Solution

Afin de résoudre ce problème, on peut détailler le raisonnement comme suit:

1. Calculer l'amplitude de chaque série L_i avec $1 \leq i \leq n$ calculer $\max(L_i, k_i) - \min(L_i, k_i)$
2. Stocker chaque amplitude dans une nouvelle série R de n valeurs
3. Calculer l'amplitude maximale de la série R au moyen de la fonction $\max(R, n)$

Les fonctions sont réutilisables

Les fonctions permettent de réutiliser des blocs d'instructions pour résoudre des problèmes similaires

Pourquoi les fonctions ?

Mais encore. . .

Lorsqu'il s'agira d'implémenter un algorithme dans un langage de programmation, les fonctions permettent de:

Pourquoi les fonctions ?

Mais encore. . .

Lorsqu'il s'agira d'implémenter un algorithme dans un langage de programmation, les fonctions permettent de:

- Rendre les programmes **robustes** et **facile à maintenir** : chaque fonction est un bloc indépendant qui peut être testé et modifiés séparément.

Pourquoi les fonctions ?

Mais encore. . .

Lorsqu'il s'agira d'implémenter un algorithme dans un langage de programmation, les fonctions permettent de:

- Rendre les programmes **robustes** et **facile à maintenir** : chaque fonction est un bloc indépendant qui peut être testé et modifiés séparément.
- Améliorer la **lisibilité** du code : les fonctions permettent de découper un programme en blocs plus petits et plus faciles à comprendre.

Pourquoi les fonctions ?

Mais encore. . .

Lorsqu'il s'agira d'implémenter un algorithme dans un langage de programmation, les fonctions permettent de:

- Rendre les programmes **robustes** et **facile à maintenir** : chaque fonction est un bloc indépendant qui peut être testé et modifiés séparément.
- Améliorer la **lisibilité** du code : les fonctions permettent de découper un programme en blocs plus petits et plus faciles à comprendre.

Pourquoi les fonctions ?

Mais encore. . .

Lorsqu'il s'agira d'implémenter un algorithme dans un langage de programmation, les fonctions permettent de:

- Rendre les programmes **robustes** et **facile à maintenir** : chaque fonction est un bloc indépendant qui peut être testé et modifiés séparément.
- Améliorer la **lisibilité** du code : les fonctions permettent de découper un programme en blocs plus petits et plus faciles à comprendre.

En somme, les fonctions permettent de:

- **Découper un problème** en sous-problèmes plus simples

Pourquoi les fonctions ?

Mais encore. . .

Lorsqu'il s'agira d'implémenter un algorithme dans un langage de programmation, les fonctions permettent de:

- Rendre les programmes **robustes** et **facile à maintenir** : chaque fonction est un bloc indépendant qui peut être testé et modifiés séparément.
- Améliorer la **lisibilité** du code : les fonctions permettent de découper un programme en blocs plus petits et plus faciles à comprendre.

En somme, les fonctions permettent de:

- **Découper un problème** en sous-problèmes plus simples
- **Organiser le raisonnement** et la résolution d'un problème

Pourquoi les fonctions ?

Mais encore. . .

Lorsqu'il s'agira d'implémenter un algorithme dans un langage de programmation, les fonctions permettent de:

- Rendre les programmes **robustes** et **facile à maintenir** : chaque fonction est un bloc indépendant qui peut être testé et modifiés séparément.
- Améliorer la **lisibilité** du code : les fonctions permettent de découper un programme en blocs plus petits et plus faciles à comprendre.

En somme, les fonctions permettent de:

- **Découper un problème** en sous-problèmes plus simples
- **Organiser le raisonnement** et la résolution d'un problème
- **Réutiliser** des blocs d'instructions à différents endroits du programme

Pourquoi les fonctions ?

Mais encore. . .

Lorsqu'il s'agira d'implémenter un algorithme dans un langage de programmation, les fonctions permettent de:

- Rendre les programmes **robustes** et **facile à maintenir** : chaque fonction est un bloc indépendant qui peut être testé et modifiés séparément.
- Améliorer la **lisibilité** du code : les fonctions permettent de découper un programme en blocs plus petits et plus faciles à comprendre.

En somme, les fonctions permettent de:

- **Découper un problème** en sous-problèmes plus simples
- **Organiser le raisonnement** et la résolution d'un problème
- **Réutiliser** des blocs d'instructions à différents endroits du programme
- Rendre le code plus **robuste** et **lisible**

Pourquoi les fonctions ?

Mais encore. . .

Lorsqu'il s'agira d'implémenter un algorithme dans un langage de programmation, les fonctions permettent de:

- Rendre les programmes **robustes** et **facile à maintenir** : chaque fonction est un bloc indépendant qui peut être testé et modifiés séparément.
- Améliorer la **lisibilité** du code : les fonctions permettent de découper un programme en blocs plus petits et plus faciles à comprendre.

En somme, les fonctions permettent de:

- **Découper un problème** en sous-problèmes plus simples
- **Organiser le raisonnement** et la résolution d'un problème
- **Réutiliser** des blocs d'instructions à différents endroits du programme
- Rendre le code plus **robuste** et **lisible**
- **Travailler à plusieurs** sur un même problème

Déclaration d'une fonction (Prototype)

Définition

Le **prototype** d'une fonction est une déclaration qui permet de faire connaître l'existence de la fonction et de renseigner les informations la concernant:

Déclaration d'une fonction (Prototype)

Définition

Le **prototype** d'une fonction est une déclaration qui permet de faire connaître l'existence de la fonction et de renseigner les informations la concernant:

- son *nom*

Déclaration d'une fonction (Prototype)

Définition

Le **prototype** d'une fonction est une déclaration qui permet de faire connaître l'existence de la fonction et de renseigner les informations la concernant:

- son *nom*
- ses *paramètres d'entrées* : lister les types et les noms des paramètres que la fonction attend

Déclaration d'une fonction (Prototype)

Définition

Le **prototype** d'une fonction est une déclaration qui permet de faire connaître l'existence de la fonction et de renseigner les informations la concernant:

- son *nom*
- ses *paramètres d'entrées* : lister les types et les noms des paramètres que la fonction attend
- ses *paramètres de sorties* : renseigner le type de la valeur que la fonction renvoie (s'il y en a)

Déclaration d'une fonction (Prototype)

Définition

Le **prototype** d'une fonction est une déclaration qui permet de faire connaître l'existence de la fonction et de renseigner les informations la concernant:

- son *nom*
- ses *paramètres d'entrées* : lister les types et les noms des paramètres que la fonction attend
- ses *paramètres de sorties* : renseigner le type de la valeur que la fonction renvoie (s'il y en a)

Déclaration d'une fonction (Prototype)

Définition

Le **prototype** d'une fonction est une déclaration qui permet de faire connaître l'existence de la fonction et de renseigner les informations la concernant:

- son *nom*
- ses *paramètres d'entrées* : lister les types et les noms des paramètres que la fonction attend
- ses *paramètres de sorties* : renseigner le type de la valeur que la fonction renvoie (s'il y en a)

Syntaxe

```
Fonction: nom (param1: type1, param2: type2, ...) -> type_retour
```

Déclaration d'une fonction (Prototype)

Définition

Le **prototype** d'une fonction est une déclaration qui permet de faire connaître l'existence de la fonction et de renseigner les informations la concernant:

- son *nom*
- ses *paramètres d'entrées* : lister les types et les noms des paramètres que la fonction attend
- ses *paramètres de sorties* : renseigner le type de la valeur que la fonction renvoie (s'il y en a)

Syntaxe

```
Fonction: nom (param1: type1, param2: type2, ...) -> type_retour
```

Exemple

```
Fonction: max (L: Tableau d'entier, k: entier) -> entier
```

Déclaration d'une fonction (Prototype)

Remarques

- Le **nom** de la fonction doit suivre la même convention de nommage que les noms de variables.

Déclaration d'une fonction (Prototype)

Remarques

- Le **nom** de la fonction doit suivre la même convention de nommage que les noms de variables.
- Les **paramètres d'entrées** sont *passés* à la fonction lors de son appel.

Déclaration d'une fonction (Prototype)

Remarques

- Le **nom** de la fonction doit suivre la même convention de nommage que les noms de variables.
- Les **paramètres d'entrées** sont *passés* à la fonction lors de son appel.
- Le **type de retour** est le type de la valeur que la fonction renvoie. Il se peut qu'une fonction ne renvoie rien, dans ce cas (cas d'un affichage, par exemple).

Déclaration d'une fonction (Prototype)

Remarques

- Le **nom** de la fonction doit suivre la même convention de nommage que les noms de variables.
- Les **paramètres d'entrées** sont *passés* à la fonction lors de son appel.
- Le **type de retour** est le type de la valeur que la fonction renvoie. Il se peut qu'une fonction ne renvoie rien, dans ce cas (cas d'un affichage, par exemple).
- Il est également possible qu'une fonction n'ait pas de paramètres d'entrées.

Définition d'une fonction (Corps)

- Définir une fonction, c'est décrire l'ensemble des instructions que la fonction aura a exécuter lors de son *appel*.

Définition d'une fonction (Corps)

- Définir une fonction, c'est décrire l'ensemble des instructions que la fonction aura a exécuter lors de son *appel*.
- L'ensemble des instructions délimité par un bloc `Début` et `Fin` est appelé le **corps** de la fonction.

Définition d'une fonction (Corps)

- Définir une fonction, c'est décrire l'ensemble des instructions que la fonction aura a exécuter lors de son *appel*.
- L'ensemble des instructions délimité par un bloc `Début` et `Fin` est appelé le **corps** de la fonction.

Définition d'une fonction (Corps)

- Définir une fonction, c'est décrire l'ensemble des instructions que la fonction aura a exécuter lors de son *appel*.
- L'ensemble des instructions délimité par un bloc `Début` et `Fin` est appelé le **corps** de la fonction.

Syntaxe

```
Fonction: nom (param1: type1, param2: type2, ...) -> type_retour
Variables locales: (* Déclaration des variables locales *)
    var1: type1
    var2: type2
    ...
Début
    (* Instructions *)
Fin
```

Définition d'une fonction (Corps)

Remarques

- Les **variables locales** sont des variables qui sont utilisées uniquement à l'intérieur de la fonction, et sont uniquement connues de la fonction.

Définition d'une fonction (Corps)

Remarques

- Les **variables locales** sont des variables qui sont utilisées uniquement à l'intérieur de la fonction, et sont uniquement connues de la fonction.
- Les paramètres d'entrées sont des variables locales qui sont initialisées avec les valeurs passées lors de l'appel de la fonction : **ils ne sont pas à déclarés**.

Définition d'une fonction (Corps)

Remarques

- Les **variables locales** sont des variables qui sont utilisées uniquement à l'intérieur de la fonction, et sont uniquement connues de la fonction.
- Les paramètres d'entrées sont des variables locales qui sont initialisées avec les valeurs passées lors de l'appel de la fonction : **ils ne sont pas à déclarés**.
- Les variables locales sont détruites à la fin de l'exécution de la fonction.

Exemple

```
Fonction: max (L: Tableau d'entier, k: entier) → entier (*1*)
Variables locales:
  i, val_max: entier
Début
  val_max ← L[1]
  Pour i ← 2 à k  Faire
      Si L[i] > val_max  Alors
          val_max ← L[i]
      Fin Si
  Fin Pour
  Retourner val_max
Fin
```

Exemple

```
Fonction: max (L: Tableau d'entier, k: entier) → entier (*1*)
Variables locales:
   i, val_max: entier
Début
   val_max ← L[1]
   Pour i ← 2 à k  Faire
         Si L[i] > val_max  Alors
               val_max ← L[i]
         Fin Si
   Fin Pour
   Retourner val_max
Fin
```

- (*1*) : Déclaration du prototype de la fonction `max`

Exemple

```
Fonction: max (L: Tableau d'entier, k: entier) → entier (*1*)
Variables locales:
   i, val_max: entier
Début(*2*)
   val_max ← L[1]
   Pour i ← 2 à k  Faire
         Si L[i] > val_max  Alors
               val_max ← L[i]
         Fin Si
   Fin Pour
   Retourner val_max
Fin
```

- (*1*) : Déclaration du prototype de la fonction `max`
- (*2*) : Définition du corps de la fonction `max`

Exemple

```
Fonction: max (L: Tableau d'entier, k: entier) → entier (*1*)
Variables locales:
   i, val_max: entier
Début(*2*)
   val_max ← L[1]
   Pour i ← 2 à k  Faire
         Si L[i] > val_max  Alors
               val_max ← L[i]
         Fin Si
   Fin Pour
   Retourner val_max (*3*)
Fin
```

- (*1*) : Déclaration du prototype de la fonction `max`
- (*2*) : Définition du corps de la fonction `max`
- (*3*) : La fonction renvoie la valeur `val_max` qui est de même type que le type de retour de la fonction

Remarques

- L'instruction `Retourner` permet de renvoyer une valeur à l'appelant de la fonction.

Remarques

- L'instruction `Retourner` permet de renvoyer une valeur à l'appelant de la fonction.
- L'instruction `Retourner` est optionnelle : une fonction peut ne pas renvoyer de valeur.

Remarques

- L'instruction `Retourner` permet de renvoyer une valeur à l'appelant de la fonction.
- L'instruction `Retourner` est optionnelle : une fonction peut ne pas renvoyer de valeur.
- Une fonction peut contenir plusieurs instructions `Retourner` mais une seule sera exécutée.

Remarques

- L'instruction `Retourner` permet de renvoyer une valeur à l'appelant de la fonction.
- L'instruction `Retourner` est optionnelle : une fonction peut ne pas renvoyer de valeur.
- Une fonction peut contenir plusieurs instructions `Retourner` mais une seule sera exécutée.
- L'instruction `Retourner` met fin à l'exécution de la fonction.

Remarques

- L'instruction `Retourner` permet de renvoyer une valeur à l'appelant de la fonction.
- L'instruction `Retourner` est optionnelle : une fonction peut ne pas renvoyer de valeur.
- Une fonction peut contenir plusieurs instructions `Retourner` mais une seule sera exécutée.
- L'instruction `Retourner` met fin à l'exécution de la fonction.
- Dans le cadre d'une fonction qui renvoie une valeur, le mot clé `Fin` sert alors à délimiter le corps de la fonction.

Appel d'une fonction

- Appeler une fonction revient à lui *passer* les paramètres d'entrées et à récupérer la valeur de retour si la fonction en renvoie une.

Appel d'une fonction

- Appeler une fonction revient à lui *passer* les paramètres d'entrées et à récupérer la valeur de retour si la fonction en renvoie une.
- L'appel d'une fonction se fait soit dans le corps d'une autre fonction, soit dans le corps du programme principal.

Appel d'une fonction

- Appeler une fonction revient à lui *passer* les paramètres d'entrées et à récupérer la valeur de retour si la fonction en renvoie une.
- L'appel d'une fonction se fait soit dans le corps d'une autre fonction, soit dans le corps du programme principal.
- Les paramètres passés lors de l'appel sont appelés des **arguments**.

Appel d'une fonction

- Appeler une fonction revient à lui *passer* les paramètres d'entrées et à récupérer la valeur de retour si la fonction en renvoie une.
- L'appel d'une fonction se fait soit dans le corps d'une autre fonction, soit dans le corps du programme principal.
- Les paramètres passés lors de l'appel sont appelés des **arguments**.
- Les types des paramètres passés lors de l'appel doivent correspondre aux types déclarés dans le prototype de la fonction.

Appel d'une fonction

- Appeler une fonction revient à lui *passer* les paramètres d'entrées et à récupérer la valeur de retour si la fonction en renvoie une.
- L'appel d'une fonction se fait soit dans le corps d'une autre fonction, soit dans le corps du programme principal.
- Les paramètres passés lors de l'appel sont appelés des **arguments**.
- Les types des paramètres passés lors de l'appel doivent correspondre aux types déclarés dans le prototype de la fonction.
- Les valeurs des arguments seront copiées dans les paramètres dans l'ordre.

Appel d'une fonction

- Appeler une fonction revient à lui *passer* les paramètres d'entrées et à récupérer la valeur de retour si la fonction en renvoie une.
- L'appel d'une fonction se fait soit dans le corps d'une autre fonction, soit dans le corps du programme principal.
- Les paramètres passés lors de l'appel sont appelés des **arguments**.
- Les types des paramètres passés lors de l'appel doivent correspondre aux types déclarés dans le prototype de la fonction.
- Les valeurs des arguments seront copiées dans les paramètres dans l'ordre.

Appel d'une fonction

- Appeler une fonction revient à lui *passer* les paramètres d'entrées et à récupérer la valeur de retour si la fonction en renvoie une.
- L'appel d'une fonction se fait soit dans le corps d'une autre fonction, soit dans le corps du programme principal.
- Les paramètres passés lors de l'appel sont appelés des **arguments**.
- Les types des paramètres passés lors de l'appel doivent correspondre aux types déclarés dans le prototype de la fonction.
- Les valeurs des arguments seront copiées dans les paramètres dans l'ordre.

Syntaxe

```
nom_fonction(arg1, arg2, ...)
```

Exemple 1

On considère le programme suivant, en considérant que les fonctions `max` et `min` ont été définies au préalable:

Algorithme: CALCULER_AMPLITUDE
Variables :
  $L[5]$: Tableau d'entier
  val_max, val_min, range: entier
Début
  1: $L \leftarrow \{1, 2, 3, 4, 5\}$
  2: val_max $\leftarrow$ max$(L, 5)$
  3: val_min $\leftarrow$ min$(L, 5)$
  4: range $\leftarrow$ val_max - val_min
  5: Ecrire("L'amplitude de la série L est", range, "\NL")
Fin

Exemple 1

On considère le programme suivant, en considérant que les fonctions `max` et `min` ont été définies au préalable:

Algorithme: CALCULER_AMPLITUDE
Variables :
  $L[5]$: Tableau d'entier
  val_max, val_min, range: entier
Début
  1: $L \leftarrow \{1, 2, 3, 4, 5\}$
  2: val_max $\leftarrow$ max$(L, 5)$
  3: val_min $\leftarrow$ min$(L, 5)$
  4: range $\leftarrow$ val_max - val_min
  5: Ecrire("L'amplitude de la série L est", range, "\NL")
Fin

- Les fonctions `max` et `min` sont appelées dans le corps de l'algorithme (dans un programme principal).

Exemple 1

On considère le programme suivant, en considérant que les fonctions `max` et `min` ont été définies au préalable:

Algorithme: CALCULER_AMPLITUDE
Variables :
  $L[5]$: Tableau d'entier
  val_max, val_min, range: entier
Début
  1: $L \leftarrow \{1, 2, 3, 4, 5\}$
  2: val_max $\leftarrow$ max$(L, 5)$
  3: val_min $\leftarrow$ min$(L, 5)$
  4: range $\leftarrow$ val_max - val_min
  5: Ecrire("L'amplitude de la série L est", range, "\NL")
Fin

- Les fonctions `max` et `min` sont appelées dans le corps de l'algorithme (dans un programme principal).
- À la ligne 2, la fonction `max` est appelée avec les arguments `L` et 5, et la valeur retournée est stockée dans la variable `val_max`.

Exemple 1

On considère le programme suivant, en considérant que les fonctions `max` et `min` ont été définies au préalable:

Algorithme: CALCULER_AMPLITUDE
Variables :
  $L[5]$: Tableau d'entier
  val_max, val_min, range: entier
Début
  1: $L \leftarrow \{1, 2, 3, 4, 5\}$
  2: val_max $\leftarrow$ max$(L, 5)$
  3: val_min $\leftarrow$ min$(L, 5)$
  4: range $\leftarrow$ val_max - val_min
  5: Ecrire("L'amplitude de la série L est", range, "\NL")
Fin

- Les fonctions `max` et `min` sont appelées dans le corps de l'algorithme (dans un programme principal).
- À la ligne 2, la fonction `max` est appelée avec les arguments `L` et 5, et la valeur retournée est stockée dans la variable `val_max`.
- À la ligne 3, la fonction `min` est appelée avec les arguments `L` et 5, et la valeur retournée est stockée dans la variable `val_min`.

Exemple 2

- En réalité, l'ensemble des instructions de l'algorithme précédent peut être encapsulé dans une fonction `amplitude` qui prendra en paramètres:

Exemple 2

- En réalité, l'ensemble des instructions de l'algorithme précédent peut être encapsulé dans une fonction `amplitude` qui prendra en paramètres:
  - Un tableau d'entier L

Exemple 2

- En réalité, l'ensemble des instructions de l'algorithme précédent peut être encapsulé dans une fonction `amplitude` qui prendra en paramètres:
  - Un tableau d'entier L
  - Un entier k représentant la taille du tableau k

Exemple 2

- En réalité, l'ensemble des instructions de l'algorithme précédent peut être encapsulé dans une fonction `amplitude` qui prendra en paramètres:
  - Un tableau d'entier L
  - Un entier k représentant la taille du tableau k

Exemple 2

- En réalité, l'ensemble des instructions de l'algorithme précédent peut être encapsulé dans une fonction `amplitude` qui prendra en paramètres:
  - Un tableau d'entier L
  - Un entier k représentant la taille du tableau k

Fonction: amplitude (L: Tableau d'entier, k: entier) $\rightarrow$ entier
Variables locales: vide
  Retourner max(L, k) - min(L, k)
Fin

- Les fonction `max` et `min` sont appelées dans le corps de la fonction `amplitude`.

Exemple 2

- En réalité, l'ensemble des instructions de l'algorithme précédent peut être encapsulé dans une fonction `amplitude` qui prendra en paramètres:
  - Un tableau d'entier L
  - Un entier k représentant la taille du tableau k

Fonction: amplitude (L: Tableau d'entier, k: entier) $\rightarrow$ entier
Variables locales: vide
  Retourner max(L, k) - min(L, k)
Fin

- Les fonction `max` et `min` sont appelées dans le corps de la fonction `amplitude`.
- Ici les valeurs de retour des fonctions `max` et `min` sont directement utilisées pour calculer l'amplitude de la série L.

Exemple 2

- En réalité, l'ensemble des instructions de l'algorithme précédent peut être encapsulé dans une fonction `amplitude` qui prendra en paramètres:
  - Un tableau d'entier L
  - Un entier k représentant la taille du tableau k

Fonction: amplitude (L: Tableau d'entier, k: entier) $\rightarrow$ entier
Variables locales: vide
  Retourner max(L, k) - min(L, k)
Fin

- Les fonction `max` et `min` sont appelées dans le corps de la fonction `amplitude`.
- Ici les valeurs de retour des fonctions `max` et `min` sont directement utilisées pour calculer l'amplitude de la série L.
- **Une ligne d'instruction comportant un appel de fonction peut être considérée comme comportant une expression nécessitant d'être évaluée**.

Exemple 3

Dans cet exemple, nous allons proposer un programme principal permettant de calculer l'amplitude maximale parmi $n = 3$ séries de valeurs entières ayant respectivement $k = 5$ valeurs. (On peut donc considérer L comme étant une matrice 3×5)

Exemple 3

Dans cet exemple, nous allons proposer un programme principal permettant de calculer l'amplitude maximale parmi $n = 3$ séries de valeurs entières ayant respectivement $k = 5$ valeurs. (On peut donc considérer L comme étant une matrice 3×5)

Algorithme: CALCULER_AMPLITUDE_MAX
Constantes:
  $N = 3, K = 5$
Variables :
  $L[N][K]$: Tableau d'entier
  $R[N]$: Tableau d'entier
Début
  1: $L \leftarrow \{\{1, 2, 3, 4, 5\},$
      $\{1, 3, 5, 7, 9\},$
      $\{1, 1, 2, 3, 5\}\}$
  2: **Pour** $i \leftarrow 1$ **à** N **Faire**
  3: $R[i] \leftarrow$ amplitude$(L[i], K)$
  4: **Fin Pour**
  5: Ecrire("L'amplitude maximale parmi les séries est", max(R, N), "\NL")
Fin

Appel d'une fonction

Appel par valeur

- Lorsqu'on appelle une fonction, les valeurs des arguments sont copiées dans les paramètres de la fonction.

Appel d'une fonction

Appel par valeur

- Lorsqu'on appelle une fonction, les valeurs des arguments sont copiées dans les paramètres de la fonction.
- On parle alors d'un *appel par valeur* ou d'un *passage par valeur*.

Appel d'une fonction

Appel par valeur

- Lorsqu'on appelle une fonction, les valeurs des arguments sont copiées dans les paramètres de la fonction.
- On parle alors d'un *appel par valeur* ou d'un *passage par valeur*.

Appel d'une fonction

Appel par valeur

- Lorsqu'on appelle une fonction, les valeurs des arguments sont copiées dans les paramètres de la fonction.
- On parle alors d'un *appel par valeur* ou d'un *passage par valeur*.

Exemple

Dans la fonction max, les valeurs du tableau L et de l'entier k sont copiées dans les paramètres de la fonction et devient des variables locales.

Appel d'une fonction

Une copie ?

- Une copie d'une variable est une nouvelle variable qui a la même valeur que la variable originale **mais à une emplacement mémoire différente**.

Appel d'une fonction

Une copie ?

- Une copie d'une variable est une nouvelle variable qui a la même valeur que la variable originale **mais à une emplacement mémoire différente**.
- **Attention** : les modifications apportées à une copie d'une variable ne sont pas répercutées sur la variable originale.

Appel d'une fonction

Une copie ?

- Une copie d'une variable est une nouvelle variable qui a la même valeur que la variable originale **mais à une emplacement mémoire différente**.
- **Attention** : les modifications apportées à une copie d'une variable ne sont pas répercutées sur la variable originale.

Appel d'une fonction

Une copie ?

- Une copie d'une variable est une nouvelle variable qui a la même valeur que la variable originale **mais à une emplacement mémoire différente**.
- **Attention** : les modifications apportées à une copie d'une variable ne sont pas répercutées sur la variable originale.

Petit retour sur les variables

- Une variable est un nom donné à un espace mémoire qui contient une valeur.

Appel d'une fonction

Une copie ?

- Une copie d'une variable est une nouvelle variable qui a la même valeur que la variable originale **mais à une emplacement mémoire différente**.
- **Attention** : les modifications apportées à une copie d'une variable ne sont pas répercutées sur la variable originale.

Petit retour sur les variables

- Une variable est un nom donné à un espace mémoire qui contient une valeur.
- Une variable est caractérisée par:

Appel d'une fonction

Une copie ?

- Une copie d'une variable est une nouvelle variable qui a la même valeur que la variable originale **mais à une emplacement mémoire différente**.
- **Attention** : les modifications apportées à une copie d'une variable ne sont pas répercutées sur la variable originale.

Petit retour sur les variables

- Une variable est un nom donné à un espace mémoire qui contient une valeur.
- Une variable est caractérisée par:
  - Son nom

Appel d'une fonction

Une copie ?

- Une copie d'une variable est une nouvelle variable qui a la même valeur que la variable originale **mais à une emplacement mémoire différente**.
- **Attention** : les modifications apportées à une copie d'une variable ne sont pas répercutées sur la variable originale.

Petit retour sur les variables

- Une variable est un nom donné à un espace mémoire qui contient une valeur.
- Une variable est caractérisée par:
  - Son nom
  - Son type

Appel d'une fonction

Une copie ?

- Une copie d'une variable est une nouvelle variable qui a la même valeur que la variable originale **mais à une emplacement mémoire différente**.
- **Attention** : les modifications apportées à une copie d'une variable ne sont pas répercutées sur la variable originale.

Petit retour sur les variables

- Une variable est un nom donné à un espace mémoire qui contient une valeur.
- Une variable est caractérisée par:
  - Son nom
  - Son type
  - Sa valeur

Appel d'une fonction

Une copie ?

- Une copie d'une variable est une nouvelle variable qui a la même valeur que la variable originale **mais à une emplacement mémoire différente**.
- **Attention** : les modifications apportées à une copie d'une variable ne sont pas répercutées sur la variable originale.

Petit retour sur les variables

- Une variable est un nom donné à un espace mémoire qui contient une valeur.
- Une variable est caractérisée par:
  - Son nom
  - Son type
  - Sa valeur
  - Son adresse mémoire (l'endroit où elle est stockée en mémoire)

Appel d'une fonction

- On considère la fonction `incrementer` suivante qui prend en paramètre un entier x et qui incrémente sa valeur de 1; ainsi que le programme principal suivant:

Fonction: incrementer (x: entier)
Variables locales: vide
Début
  $x \leftarrow x + 1$
Fin

Algorithme: MAIN
Variables :
  a: entier
Début
  $a \leftarrow 5$
  incrementer(a)
  Ecrire("La valeur de a est", a, "\NL")
Fin

Appel d'une fonction

- On considère la fonction `incrementer` suivante qui prend en paramètre un entier x et qui incrémente sa valeur de 1; ainsi que le programme principal suivant:

Fonction: incrementer (x: entier)
Variables locales: vide
Début
  $x \leftarrow x + 1$
Fin

Algorithme: MAIN
Variables :
  a: entier
Début
  $a \leftarrow 5$
  incrementer(a)
  Ecrire("La valeur de a est", a, "\NL")
Fin

Appel d'une fonction

- On considère la fonction `incrementer` suivante qui prend en paramètre un entier x et qui incrémente sa valeur de 1; ainsi que le programme principal suivant:

Fonction: incrementer (x: entier)
Variables locales: vide
Début
  $x \leftarrow x + 1$
Fin

Algorithme: MAIN
Variables :
  a: entier
Début
  $a \leftarrow 5$
  incrementer(a)
  Ecrire("La valeur de a est", a, "\NL")
Fin

Question

Que va afficher le programme principal ?

Appel d'une fonction

- On considère la fonction `incrementer` suivante qui prend en paramètre un entier x et qui incrémente sa valeur de 1; ainsi que le programme principal suivant:

Fonction: incrementer (x: entier)
Variables locales: vide
Début
  $x \leftarrow x + 1$
Fin

Algorithme: MAIN
Variables :
  a: entier
Début
  $a \leftarrow 5$
  incrementer(a)
  Ecrire("La valeur de a est", a, "\NL")
Fin

Question

Que va afficher le programme principal ?

Réponse

Le programme affichera 5 car la fonction `incrementer` reçoit une copie de la variable a et non la variable a elle-même.

Appel d'une fonction

Appel par référence

- Pour permettre à une fonction de modifier la valeur d'une variable passée en argument, on peut utiliser un *appel par référence*.

Appel d'une fonction

Appel par référence

- Pour permettre à une fonction de modifier la valeur d'une variable passée en argument, on peut utiliser un *appel par référence*.
- Lors d'un appel par référence, c'est l'adresse mémoire de la variable qui est passée à la fonction et non la valeur de la variable **afin que la fonction puisse modifier la valeur de la référencée par le paramètre**.

Appel d'une fonction

Appel par référence

- Pour permettre à une fonction de modifier la valeur d'une variable passée en argument, on peut utiliser un *appel par référence*.
- Lors d'un appel par référence, c'est l'adresse mémoire de la variable qui est passée à la fonction et non la valeur de la variable **afin que la fonction puisse modifier la valeur de la référencée par le paramètre**.
- Il est donc nécessaire de préciser que le paramètre est passé par référence lors de la déclaration de la fonction.

Appel d'une fonction

Appel par référence

- Pour permettre à une fonction de modifier la valeur d'une variable passée en argument, on peut utiliser un *appel par référence*.
- Lors d'un appel par référence, c'est l'adresse mémoire de la variable qui est passée à la fonction et non la valeur de la variable **afin que la fonction puisse modifier la valeur de la référencée par le paramètre**.
- Il est donc nécessaire de préciser que le paramètre est passé par référence lors de la déclaration de la fonction.
- Dans le cadre du cours, on convenient que **modifier une référence à une variable revient à modifier la variable elle-même.**

Appel d'une fonction

Appel par référence

- Pour permettre à une fonction de modifier la valeur d'une variable passée en argument, on peut utiliser un *appel par référence*.
- Lors d'un appel par référence, c'est l'adresse mémoire de la variable qui est passée à la fonction et non la valeur de la variable **afin que la fonction puisse modifier la valeur de la référencée par le paramètre**.
- Il est donc nécessaire de préciser que le paramètre est passé par référence lors de la déclaration de la fonction.
- Dans le cadre du cours, on convenient que **modifier une référence à une variable revient à modifier la variable elle-même.**

Appel d'une fonction

Appel par référence

- Pour permettre à une fonction de modifier la valeur d'une variable passée en argument, on peut utiliser un *appel par référence*.
- Lors d'un appel par référence, c'est l'adresse mémoire de la variable qui est passée à la fonction et non la valeur de la variable **afin que la fonction puisse modifier la valeur de la référencée par le paramètre**.
- Il est donc nécessaire de préciser que le paramètre est passé par référence lors de la déclaration de la fonction.
- Dans le cadre du cours, on convenient que **modifier une référence à une variable revient à modifier la variable elle-même.**

Notation

- On ajoute `Référence` devant le type du paramètre dans le prototype de la fonction.

Appel d'une fonction

Appel par référence

- Pour permettre à une fonction de modifier la valeur d'une variable passée en argument, on peut utiliser un *appel par référence*.
- Lors d'un appel par référence, c'est l'adresse mémoire de la variable qui est passée à la fonction et non la valeur de la variable **afin que la fonction puisse modifier la valeur de la référencée par le paramètre**.
- Il est donc nécessaire de préciser que le paramètre est passé par référence lors de la déclaration de la fonction.
- Dans le cadre du cours, on convenient que **modifier une référence à une variable revient à modifier la variable elle-même.**

Notation

- On ajoute `Référence` devant le type du paramètre dans le prototype de la fonction.
- On ajoute `&` devant le nom de la variable lors de l'appel de la fonction.

Appel d'une fonction

Appel par référence

Ainsi, la fonction `incrementer` et le programme principal deviennent:

Fonction: incrementer (x: Référence entier)
Variables locales: vide
Début
  $x \leftarrow x + 1$
Fin

Algorithme: MAIN
Variables :
  a: entier
Début
  $a \leftarrow 5$
  incrementer(&a)
  Ecrire("La valeur de a est", a, "\NL")
Fin

- Dans ce cas précis, le programme affichera 6 car la fonction `incrementer` reçoit une référence à la variable a et non la valeur de a.

Appel d'une fonction

Appel par référence

Ainsi, la fonction `incrementer` et le programme principal deviennent:

Fonction: incrementer (x: Référence entier)
Variables locales: vide
Début
  $x \leftarrow x + 1$
Fin

Algorithme: MAIN
Variables :
  a: entier
Début
  $a \leftarrow 5$
  incrementer(&a)
  Ecrire("La valeur de a est", a, "\NL")
Fin

- Dans ce cas précis, le programme affichera 6 car la fonction `incrementer` reçoit une référence à la variable a et non la valeur de a.
- Dans la fonction `incrementer`, la variable x est une référence à la variable a.

À vous de jouer

Écrire une fonction `swap` qui prend en paramètre deux entiers a et b et qui échange les valeurs des deux entiers.

Bonnes pratiques et erreurs à éviter

- On passera toujours les paramètres d'une fonction par référence si on souhaite modifier la valeur des paramètres.

Bonnes pratiques et erreurs à éviter

- On passera toujours les paramètres d'une fonction par référence si on souhaite modifier la valeur des paramètres.
- Sauf si c'est explicitement demandé, éviter de mettre l'instruction `Lire` dans le corps d'une fonction.

Bonnes pratiques et erreurs à éviter

- On passera toujours les paramètres d'une fonction par référence si on souhaite modifier la valeur des paramètres.
- Sauf si c'est explicitement demandé, éviter de mettre l'instruction `Lire` dans le corps d'une fonction.
- Les paramètres d'une fonction ne sont pas à saisir dans le corps de la fonction, elles sont déjà initialisées lors de l'appel de la fonction.

Bonnes pratiques et erreurs à éviter

- On passera toujours les paramètres d'une fonction par référence si on souhaite modifier la valeur des paramètres.
- Sauf si c'est explicitement demandé, éviter de mettre l'instruction `Lire` dans le corps d'une fonction.
- Les paramètres d'une fonction ne sont pas à saisir dans le corps de la fonction, elles sont déjà initialisées lors de l'appel de la fonction.
- Une fonction dont la tâche est d'effectuer un calcul ou une opération sur des données soit renvoyer une valeur et non se contenter d'afficher le résultat.